

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)****Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation**

|                       |                                                                                                                    |                      |                                                         |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------------------------|
| <b>Course Code:</b>   | CSA10 / CSA1063                                                                                                    | <b>Course Title:</b> | Software Engineering                                    |
| <b>CO Assessed:</b>   | CO3                                                                                                                | <b>Assessment:</b>   | Assessment Tool 1 – Code Review & Repository Evaluation |
| <b>Weightage:</b>     | 20%                                                                                                                | <b>Total Marks:</b>  | 25                                                      |
| <b>CO3 Statement:</b> | Build full-stack applications with an emphasis on containerization, version control, and collaborative development |                      |                                                         |
| <b>Student Name:</b>  | RASHMI NAURENE                                                                                                     | <b>Reg. No.:</b>     | 192521357                                               |
| <b>Date:</b>          | 09.08.2026                                                                                                         | <b>Duration:</b>     | 60 minutes                                              |

**Code of Conduct**

I \_\_\_\_\_ RASHMI NAURENE \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_ rashmi \_\_\_\_\_

**Annexure A – Code Review & Repository Evaluation****Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

**Assignment Questions****1. Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

**2. Repository Organization**

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

**3. Code Quality Review**

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

**4. Version Control Evaluation**

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

- Explain how version control supports collaborative software development.
- 5. **Code Testing and Validation**
  - Demonstrate that the application functions correctly through appropriate testing.
  - Present test cases, execution results, and screenshots as evidence.
- 6. **Repository Documentation**
  - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
  - Suggest improvements to enhance usability and maintainability.
- 7. **Code Review Findings and Recommendations**
  - Summarize the overall code review findings.
  - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

#### Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

# 1 · Problem Overview

## 1.1 Industry Context and Assigned Problem

Organizations continuously invest in employee development to remain competitive as new technologies create skill gaps across job roles. The assigned problem is the Employee Skill-Gap Analysis and Training Recommendation System: a platform intended to evaluate employee competencies, identify skill gaps against job-role requirements, recommend personalized training, track learning progress and certifications, provide HR dashboards and analytical reports, support competency-based career development, and enable data-driven training decisions.

## 1.2 Existing Repository Implementation

The repository under review is documented, via its own README, as an Employee Performance Management System — an application that manages and evaluates employee performance. Its documented functional requirements are:

| #  | Documented Functional Requirement |
|----|-----------------------------------|
| 1  | Add Employee                      |
| 2  | Edit Employee                     |
| 3  | Delete Employee                   |
| 4  | Restore Employee                  |
| 5  | Evaluate Employee                 |
| 6  | View All Employees                |
| 7  | View Trash                        |
| 8  | Employee Grades                   |
| 9  | Grade Existence Check             |
| 10 | Clear Input Fields                |
| 11 | View Project                      |
| 12 | Add Project                       |
| 13 | Delete Project                    |
| 14 | Edit Project                      |

## 1.3 Relationship Between the Assigned Problem and the Existing Implementation

The existing system provides functionality related to employee performance evaluation, grades, skills, and project management. These features provide a foundation for competency analysis, but the available documentation does not demonstrate complete implementation of personalized training recommendations, learning paths, certification tracking, or automated learning notifications.

| Documented in the Existing Repository   | Not Demonstrated by the Available Evidence  |
|-----------------------------------------|---------------------------------------------|
| Employee CRUD (add/edit/delete/restore) | Personalized training recommendation engine |

|                                                                         |                                                          |
|-------------------------------------------------------------------------|----------------------------------------------------------|
| Employee performance, attendance, soft-skill, and hard-skill evaluation | Structured, competency-vs-role-requirement gap scoring   |
| Employee grading (EmployeeGrade)                                        | Learning path sequencing / tracking                      |
| Project management (Project, ProjectDAO)                                | Certification tracking                                   |
| PostgreSQL persistence via DAO classes                                  | Automated learning notifications                         |
| Basic JUnit testing of the Employee model                               | HR analytical dashboards specific to skill-gap reporting |

*This report therefore evaluates the existing codebase and the Git/Docker work performed on it as a foundation relevant to the assigned problem, rather than claiming the assigned system has been fully built.*

## 2 · Repository Organization

The repository was obtained by forking a public GitHub repository and cloning it locally. It is a Java/Maven JavaFX application with PostgreSQL integration.

### 2.1 Confirmed Repository Contents

- pom.xml — Maven project descriptor at the repository root.
- README.md — project documentation (evaluated in detail in Section 6).
- Dockerfile, docker-compose.yml, .dockerignore — added as part of the related Git & Docker assignment work on this same repository.
- Java source files: Employee.java, EmployeeDAO.java, EmployeeDAOInterface.java, EmployeeGrade.java, HelloApplication.java, HelloController.java, Project.java, ProjectDAO.java, module-info.java.
- EmployeeTest.java — a JUnit 5 test class.

**Not verified from the available evidence:** *the exact sub-directory path of each source file (e.g., the full src/main/java/... package path), and whether resource files (FXML/CSS) are organized under a single src/main/resources directory or split further. Standard Maven convention would place production sources under src/main/java, the module descriptor at src/main/java/module-info.java, and tests under src/test/java, but this layout is not confirmed here.*

### 2.2 Organizational Assessment

Based on the confirmed contents, the repository follows a conventional Maven single-module layout: a project descriptor (pom.xml) at the root, application code grouped by role (model classes Employee/Project/EmployeeGrade, DAO classes EmployeeDAOInterface/EmployeeDAO, and a JavaFX entry point/controller pair HelloApplication/HelloController), a module descriptor for the Java Platform Module System, and a separate test class. This separation of model, data-access, and UI-controller responsibilities into distinctly named files is a reasonable, conventional organization for a small JavaFX/Maven application, and supports maintainability by making each file's role identifiable from its name alone.

```
Volume serial number is 0000005A CAAC:E044
C:\
.
├── .dockerignore
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── Dockerfile.txt
├── mvnw
├── mvnw.cmd
├── pom.xml
├── README.md
├── .idea
│   ├── .gitignore
│   ├── dataSources.xml
│   ├── encodings.xml
│   ├── misc.xml
│   ├── sqldialects.xml
│   └── vcs.xml
├── artifacts
│   └── EmployeePerformanceApp_jar.xml
├── .mvn
│   └── wrapper
│       ├── maven-wrapper.jar
│       └── maven-wrapper.properties
├── assets
│   ├── db
│   │   ├── psql1.png
│   │   ├── psql2.png
│   │   ├── psql3.png
│   │   ├── psql4.png
│   │   └── psql5.png
│   ├── edit
│   │   ├── edit1.png
│   │   ├── edit2.png
│   │   └── edit3.png
│   ├── employees
│   │   ├── Employee1.png
│   │   ├── Employee2.png
│   │   ├── Employee3.png
│   │   └── Employee4.png
│   └── grades
│       └── Grades1.png
```

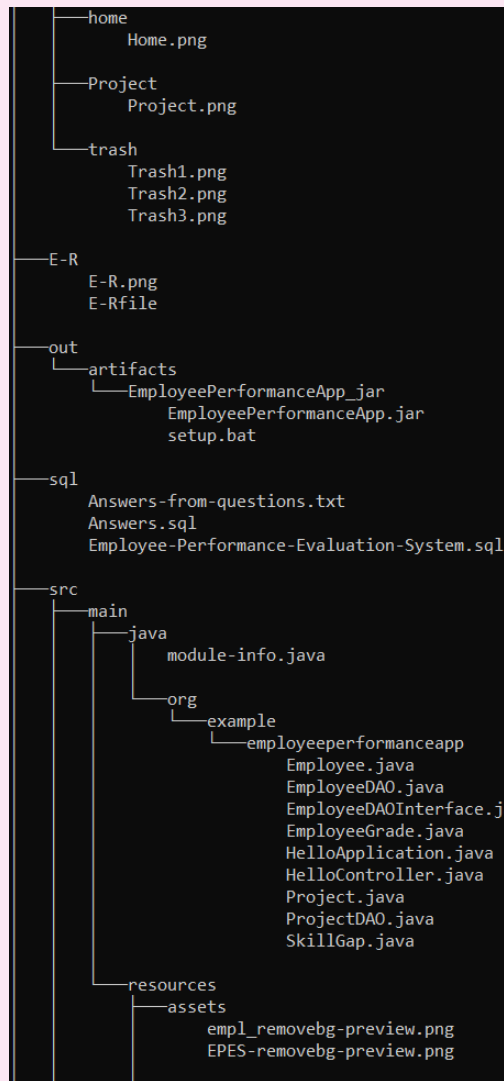


Figure 1: Project repository structure showing source, test, configuration and deployment files.

### 3 · Code Quality Review

The following actual source files were inspected for this review: Employee.java, EmployeeGrade.java, EmployeeDAO.java, HelloController.java, and EmployeeTest.java.

#### 3.1 Strengths

##### Model Encapsulation (Employee.java)

Employee.java models id, name, department, and hireDate, using JavaFX StringProperty for name and department, with a constructor, getters, setters, and JavaFX property accessor methods. This provides a clear model representation and encapsulates its properties in a way that integrates directly with JavaFX data binding.

##### Inheritance (EmployeeGrade.java)

EmployeeGrade.java extends Employee and adds a grade field with a constructor, getGrade(), and setGrade(). This is a straightforward use of inheritance to extend employee information with grade information.

## DAO Separation and Safe Query Construction (EmployeeDAO.java)

Database operations are separated into a DAO rather than being implemented inside the model classes. The DAO uses prepared statements, for example:

```
String query = "INSERT INTO employees (name, department, hire_date) VALUES (?, ?, ?)";
```

which is preferable to direct SQL string concatenation for values. The DAO also uses try-with-resources for database resources such as statements and result sets, which improves resource management by ensuring they are closed automatically.

## Testing (EmployeeTest.java)

The available tests correctly use JUnit assertions (assertEquals(...)) and cover basic Employee model behaviour, including getters/setters and the JavaFX name and department properties. Coverage detail is discussed in Section 5.

## 3.2 Areas for Improvement

### Broad Controller Responsibility (HelloController.java)

HelloController handles a wide range of unrelated responsibilities, including addEmployee(), editEmployee(), moveToTrash(), restoreEmployee(), deleteEmployee(), evaluateEmployee(), addProject(), editProject(), deleteProject(), loading employee/grade/project data, validation, and alerts. This is a finding of high responsibility concentration in a single class.

Recommendation: introduce service-layer classes, for example HelloController → EmployeeService → EmployeeDAO, and HelloController → ProjectService → ProjectDAO, to improve separation of concerns and testability.

### Hard-Coded Database Credentials (HelloController.java)

The inspected controller code directly creates a PostgreSQL connection using database credentials embedded in source, similar to:

```
Connection connection = DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/postgres",
    "postgres",
    "*****" // credential redacted for this report
);
```

This is a genuine risk: it creates security exposure, makes deployment configuration and environment changes difficult, and mixes configuration with application logic. Recommendation: move database configuration to environment variables, external configuration, Docker Compose environment configuration, or a secure secrets manager for production.

### Broad DAO Responsibility (EmployeeDAO.java)

EmployeeDAO handles employee CRUD, grades, trash operations, and restoration together, which creates a relatively large DAO responsibility. Recommendation: consider future separation into EmployeeDAO, GradeDAO, and TrashDAO to improve modularity and maintainability.

### Basic Exception Handling (EmployeeDAO.java)

Several database operations use a pattern similar to:

```
catch (SQLException e) {  
    e.printStackTrace();  
}
```

Printing a stack trace is useful during development but is not, on its own, a complete production error-handling strategy. Recommendation: use structured logging and/or propagate meaningful exceptions to the appropriate application layer.

### Transaction Management

The employee-to-trash operation conceptually involves multiple database actions — inserting the employee into trash, deleting the grade, and deleting the employee. If one operation succeeds and another fails, there is a potential risk of inconsistent database state; this is presented here as a maintainability/reliability recommendation rather than a claim that an inconsistency has actually been observed. Recommendation: wrap the multi-step operation in a database transaction (BEGIN ... COMMIT, with ROLLBACK on failure).

### Naming

Some variable names are abbreviated — for example, soft-skill-related fields use a name such as sofskill. Recommendation: use descriptive names such as softSkillField, hardSkillField, and attendanceField to improve readability.

### Overloaded Alert Methods

The controller contains overloaded showAlert() methods. This is not necessarily a defect, since method overloading is valid Java practice; however, clearer naming or centralized UI notification handling could improve readability if the controller continues to grow. This is a minor observation, not a major issue.

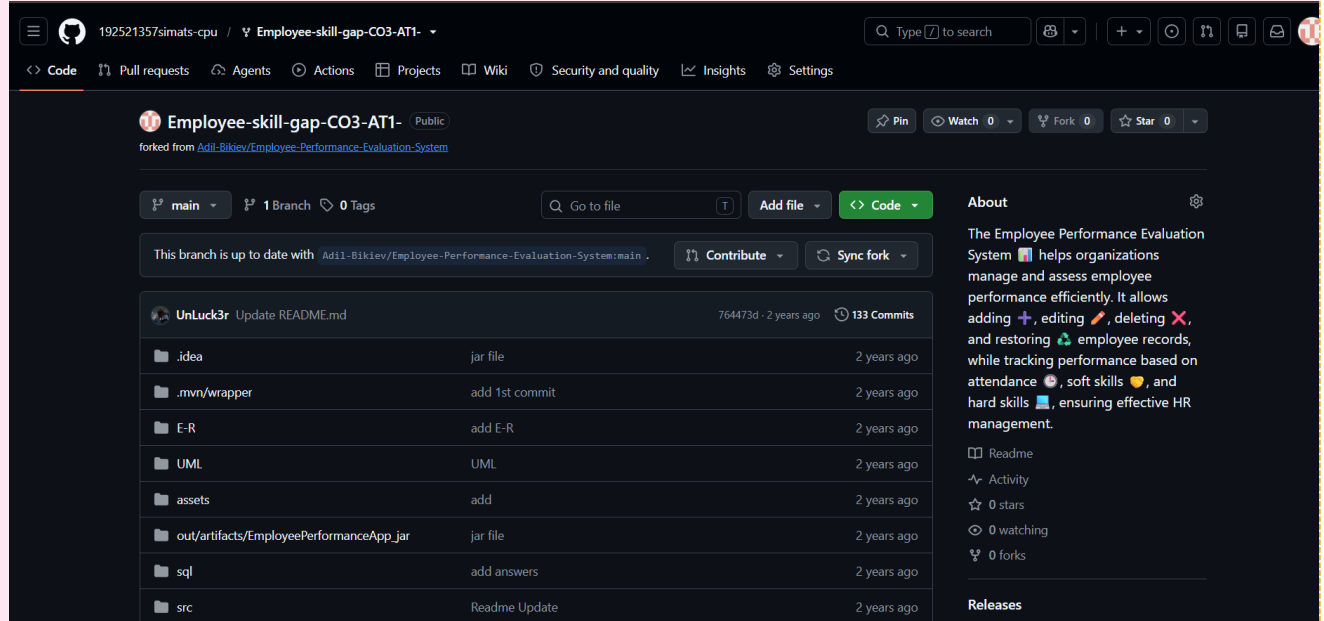
## 4 · Version Control Evaluation

---

The repository was forked from a public GitHub repository and cloned locally:

```
$ git clone https://github.com/192521357simats-cpu/Employee-skill-gap-C03-AT1-.git
```





The Git workflow used branches including main, develop, feature/skill-analysis, and feature/docker, following the flow main → develop → feature branch → commit → push → Pull Request → merge.

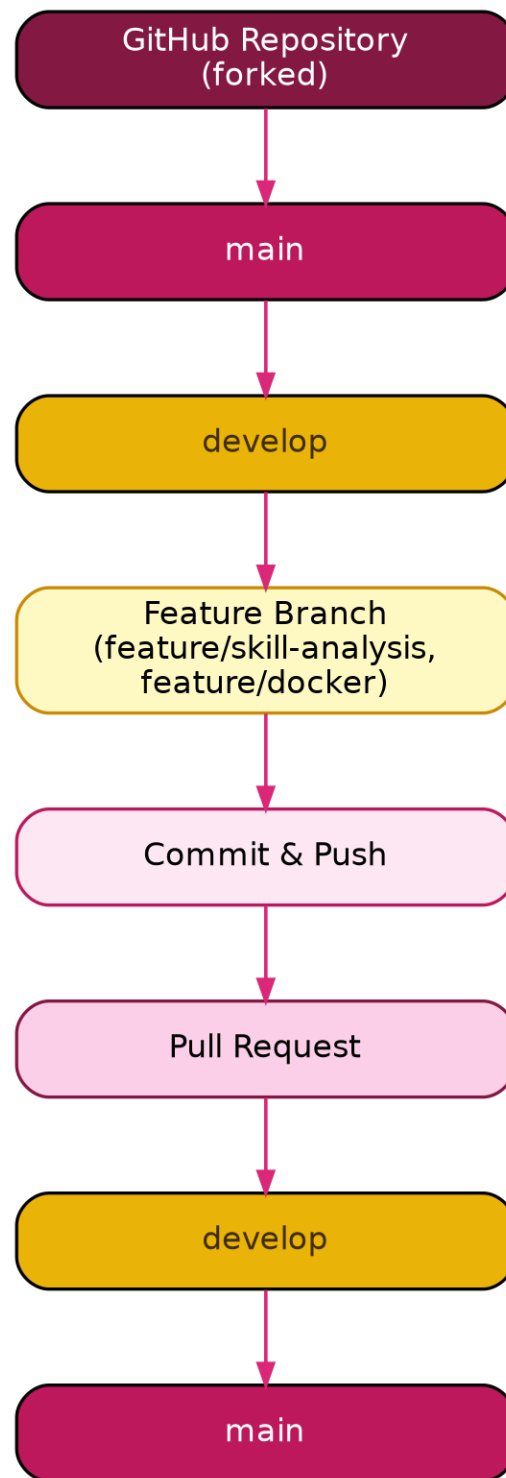


Figure 2: Branching and Pull Request workflow used for this repository.

```

main
PS C:\Windows\system32\Employee-skill-gap-CO3-AT1-> git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/192521357simats-cpu/Employee-skill-gap-CO3-AT1-/pull/new/develop
remote:
To https://github.com/192521357simats-cpu/Employee-skill-gap-CO3-AT1-.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
PS C:\Windows\system32\Employee-skill-gap-CO3-AT1-> git switch -c feature/skill-analysis
Switched to a new branch 'feature/skill-analysis'
PS C:\Windows\system32\Employee-skill-gap-CO3-AT1-> git branch
  develop
 * feature/skill-analysis
  main
PS C:\Windows\system32\Employee-skill-gap-CO3-AT1->

```

```

* 2b3314b (HEAD -> develop, origin/develop) Merge pull request #1 from 192521357simats-cpu/feature/skill-analysis
|
| * 044cb46 (origin/feature/skill-analysis, feature/skill-analysis) Add skill gap analysis model
|/
* 764473d (origin/main, origin/HEAD, main) Update README.md
* 7a85dc6 Update README.md
* 2cca87e add answers
* 556422b Readme update
* dc377cc Readme E-R
* 1af31f2 Readme ER
* dff5109 Readme Update
|
| * b05c958 add sql
| * 6df07db add
| * d198b40 Update README.md
| * bfd98d4 add Validation
| * 88e7fb0 Readme
| * 976998c Readme
|/
* 0a6829a Merge branch 'main' of https://github.com/EMMMABK/Employee-Performance-Evaluation-System
|
| * 1a8c344 Update README.md
| * e4c8051 UML
|/
* 0f505e0 Update README.md
* 783d813 Update README.md
* 4abafbe add E-R
* 01062de Update Employee-Performance-Evaluation-System.sql
* 51c79ab add one entity
* 87f5990 add new entity
* d3d42dc new Table Insert
* 6fad8f3 add 2-nd entity
* 42c00c8 add modified SQL
* 787cad9 add Alert same ID exception
* 949e6e9 Update README.md
* aace859 Update README.md
* 3348f44 Update README.md
* 8ba1f93 jar file
* c5e4555 UNIT_Test_Readme
* 0929288 UNIT_Test
* c5e6f9a add remove comments
* 1128d57 Update README.md
* fbd3d65 Update README.md
* 5013755 readme
* 61fb28e UML
* 67b489c add jar file
* d157261 add SQL updating
* f9fb050 Update README.md

```

The work included branch creation, feature development, commits, pushing branches, a Pull Request, merging, merge conflict creation/resolution, repository synchronization, and commit history inspection.

The screenshot shows a GitHub pull request titled "Add skill gap analysis model #1". The interface is in dark mode. At the top, there's a navigation bar with links: Code, Pull requests (1), Agents, Actions, Projects, Wiki, Security and quality, and More. Below this, there's a "Code" button and a "Code" dropdown. The main title "Add skill gap analysis model #1" is followed by an edit icon. A green "Open" button is next to the text "192521357simats-cpu wants to merge 1 commit into develop from feature/skill-analysis". Below this, there's a section for "Conversation" (0), "Commits" (1), "Checks" (0), and "Files changed" (1). The main content area shows a comment from "192521357simats-cpu" (Owner) stating: "Adds the SkillGap model to support employee skill-gap analysis. The model stores: Employee ID, Skill name, Required skill level, Current skill level, Calculated skill gap". To the right, there's a sidebar with sections: "Reviewers" (No reviews, Still in progress? Convert to draft), "Assignees" (No one—assign yourself), "Labels" (None yet), "Projects" (None yet), and "Milestone" (No milestone).

Code Pull requests 1 Agents Actions Projects Wiki Security and quality More

Code

# Add skill gap analysis model #1

Open 192521357simats-cpu wants to merge 1 commit into develop from feature/skill-analysis

Conversation 0 Commits 1 Checks 0 Files changed 1

 192521357simats-cpu commented now Owner ...

Adds the SkillGap model to support employee skill-gap analysis.

The model stores:

- Employee ID
- Skill name
- Required skill level
- Current skill level
- Calculated skill gap

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

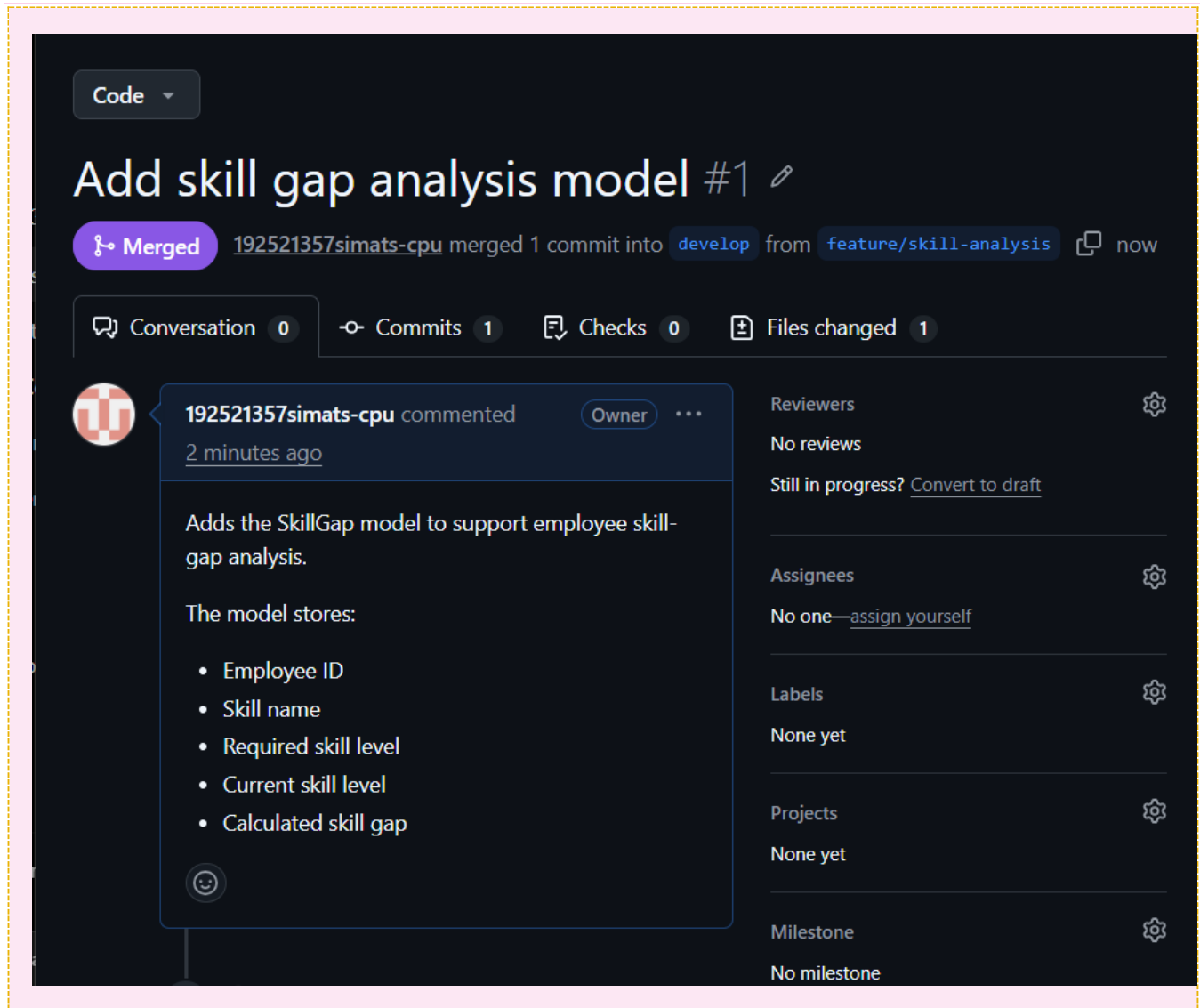
None yet

Projects

None yet

Milestone

No milestone



## 4.1 Strengths

- Feature branches isolate changes, keeping unrelated work (the skill-analysis model and the Docker setup) separable.
- develop provides a single integration branch ahead of main.
- Pull Requests provide a defined review/merge point rather than merging directly.
- Commit history provides traceability of changes.
- Merge conflict resolution was demonstrated, showing collaborative version-control handling rather than only conflict-free merges.

## 4.2 Areas for Improvement

- Commit messages should consistently use clear, action-oriented descriptions across the full history, not only the confirmed example.
- Branch naming conventions should be documented explicitly (e.g., in the README) so contributors follow a consistent feature/, release/, hotfix/ pattern.
- The README could include a contribution/branching workflow section for new contributors.

- For a larger team, a protected main branch with required Pull Request reviews could be considered.

## 5 · Code Testing and Validation

### 5.1 Test Environment and Compatibility

The project's original pom.xml targeted Java 23. The environment used for local testing had javac 21.0.11 installed. Running the test suite initially failed with the error:

```
invalid target release: 23
```

For local testing, the Maven compiler target was temporarily aligned with Java 21. After this adjustment, the following command completed successfully:

```
$ .\mvnw.cmd clean test
```

*The original Java 23 configuration is not claimed to work unmodified on a Java 21 environment; the successful run reflects the temporarily adjusted compiler target only.*

```
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-artifact-manager/2.0.9/maven-artifact-manager-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-project/2.0.9/maven-project-2.0.9.jar (122 kB at 214 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-registry/2.0.9/maven-plugin-registry-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-settings/2.0.9/maven-settings-2.0.9.jar (49 kB at 81 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-core/2.0.9/maven-core-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-profile/2.0.9/maven-profile-2.0.9.jar (35 kB at 55 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-parameter-documenter/2.0.9/maven-plugin-parameter-documenter-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-registry/2.0.9/maven-plugin-registry-2.0.9.jar (29 kB at 40 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/reporting/maven-reporting-api/2.0.9/maven-reporting-api-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-artifact-manager/2.0.9/maven-artifact-manager-2.0.9.jar (58 kB at 79 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-model/2.0.9/maven-model-2.0.9.jar (87 kB at 120 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-error-diagnostics/2.0.9/maven-error-diagnostics-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-repository-metadata/2.0.9/maven-repository-metadata-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-core/2.0.9/maven-core-2.0.9.jar (160 kB at 213 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-descriptor/2.0.9/maven-plugin-descriptor-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-parameter-documenter/2.0.9/maven-plugin-parameter-documenter-2.0.9.jar (21 kB at 26 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-monitor/2.0.9/maven-monitor-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-error-diagnostics/2.0.9/maven-error-diagnostics-2.0.9.jar (14 kB at 17 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-toolchain/2.0.9/maven-toolchain-2.0.9.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/reporting/maven-reporting-api/2.0.9/maven-reporting-api-2.0.9.jar (10 kB at 13 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugin-tools/maven-plugin-annotations/3.1/maven-plugin-annotations-3.1.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-repository-metadata/2.0.9/maven-repository-metadata-2.0.9.jar (25 kB at 30 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-descriptor/2.0.9/maven-plugin-descriptor-2.0.9.jar (37 kB at 43 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-monitor/2.0.9/maven-monitor-2.0.9.jar (10 kB at 12 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-toolchain/2.0.9/maven-toolchain-2.0.9.jar (38 kB at 43 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugin-tools/maven-plugin-annotations/3.1/maven-plugin-annotations-3.1.jar (14 kB at 16 kB/s)
[INFO] Surefire report directory: C:\Windows\System32\Employee-skill-gap-C03-AT1-Target\surefire-reports
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit3/2.12.4/surefire-junit3-2.12.4.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-providers/2.12.4/surefire-providers-2.12.4.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit3/2.12.4/surefire-junit3-2.12.4.jar (26 kB at 320 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit3/2.12.4/surefire-junit3-2.12.4.jar (26 kB at 320 kB/s)

-----
T E S T S
-----
Results :

Tests run: 0, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.790 s
[INFO] Finished at: 2026-08-09T17:06:36+05:30
[INFO]
PS C:\Windows\System32\Employee-skill-gap-C03-AT1->
```

Figure 3: Successful test execution after aligning the compiler target with the available JDK.

### 5.2 Test Cases

| Test Method             | What It Verifies                                                   | Result |
|-------------------------|--------------------------------------------------------------------|--------|
| testGettersAndSetters() | Employee ID, name, department, hire date, and setter functionality | Passed |
| testNameProperty()      | JavaFX name StringProperty behaviour                               | Passed |

|                          |                                            |        |
|--------------------------|--------------------------------------------|--------|
| testDepartmentProperty() | JavaFX department StringProperty behaviour | Passed |
|--------------------------|--------------------------------------------|--------|

### 5.3 Coverage Limitations

The available test class covers basic Employee model behaviour only. It does not demonstrate coverage of EmployeeDAO database operations, project operations, trash/restore operations, grade persistence, employee evaluation logic, controller behaviour, or integration with PostgreSQL. Recommendation: expand the test suite to include DAO-level tests (potentially against a test database), controller/service-level tests, and integration tests covering the trash/restore and grading workflows.

### 5.4 Environment Documentation Finding

The project should clearly document the required JDK version so that developers can reproduce the build environment without independently discovering the Java 23-vs-21 mismatch encountered during this review.

## 6 · Repository Documentation

The README title is Employee Performance Management System, with a table of contents covering: Description; Project Requirements List; Team Members List; Roles of Group Members; Screenshots; Sample Data; UML Class Diagram; E-R Diagrams; Weekly Meeting Documentation; OOP Concepts and Questions; Unit Test Cases; Presentation; GitHub Repository; Jar File Build; and Youtube.

The README states that the system helps organizations manage and assess employee performance, and describes support for adding, editing, deleting, moving-to-trash, and restoring employees; employee performance, attendance, soft-skill, and hard-skill evaluation; employee grades; and project management.

### 6.1 Documentation Strengths

- A table of contents is provided.
- Functional requirements are explicitly listed.
- Team members and roles are documented.
- Sample PostgreSQL data is provided.
- UML and ER diagrams are referenced.
- Unit testing is documented.
- Presentation and repository information are included.
- JAR build information is included.

### 6.2 Documentation Weaknesses

Project title mismatch: the README describes an Employee Performance Management System, while the assigned problem is the Employee Skill-Gap Analysis and Training Recommendation System.

Incomplete screenshots section: the README contains entries such as “Home:”, “Employee:”, “Edit:”, “Grades:”, “Trash:”, and “Database:”, but the provided README content does not contain the actual screenshot images.

Limited installation instructions: the README does not clearly provide a structured prerequisites section, installation procedure, database setup procedure, configuration procedure, or run instructions.

Usage documentation could be improved: the README describes functionalities but does not provide a clear step-by-step workflow for a new user.

Dependency documentation could be clearer: sample SQL is provided, but setup requirements and dependency configuration could be explained more clearly.

Encoding issue: the README contains malformed characters such as “employeeâ€™s” and “systemâ€™s”, indicating the file was likely saved or exported without consistent UTF-8 encoding (a right single quotation mark rendering as a mis-decoded byte sequence).

### 6.3 Documentation Evaluation Table

| Documentation Area | Evaluation                            |
|--------------------|---------------------------------------|
| Description        | Present                               |
| Requirements       | Present                               |
| Team information   | Present                               |
| Sample data        | Present                               |
| UML                | Present / referenced                  |
| ER diagram         | Present / referenced                  |
| Unit tests         | Present / referenced                  |
| Installation guide | Needs improvement                     |
| Usage guide        | Needs improvement                     |
| Dependency setup   | Needs improvement                     |
| Screenshots        | Section exists but appears incomplete |
| Encoding           | Needs correction                      |

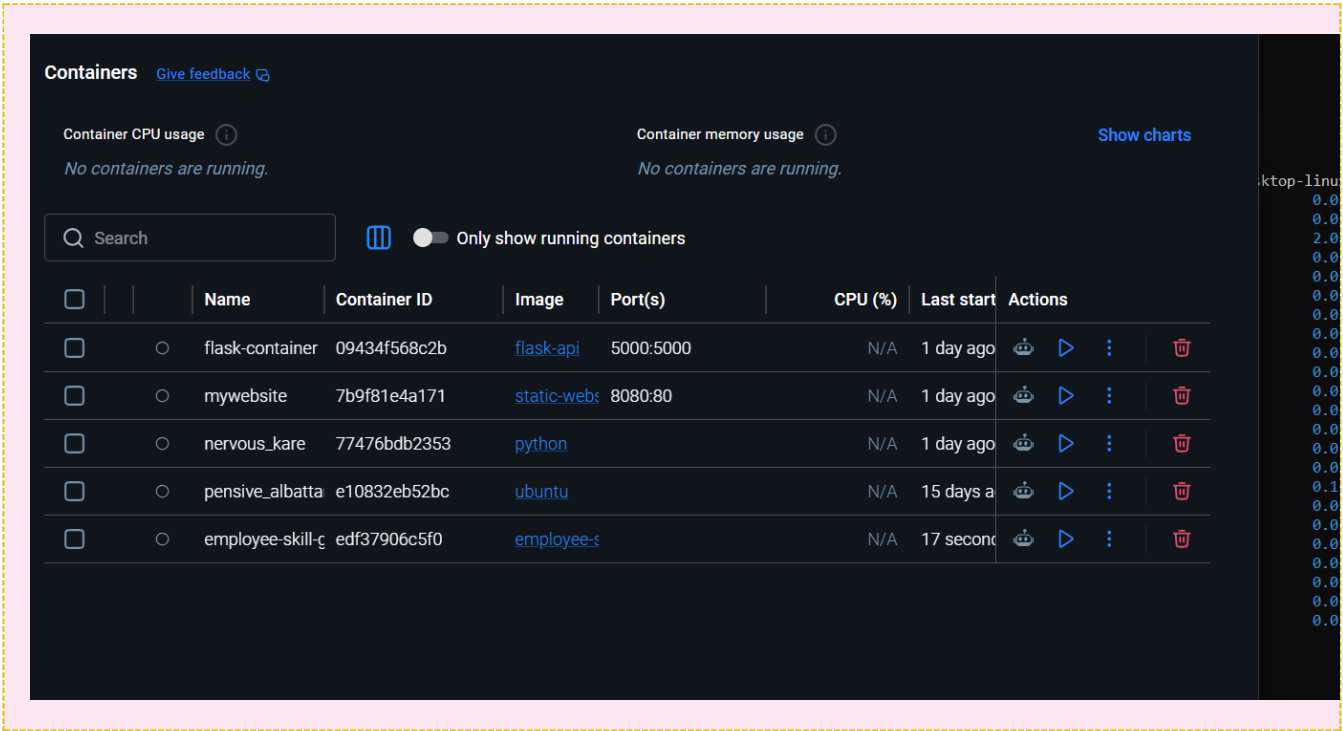
### 6.4 Recommendations

- Add a clear prerequisites section (JDK version, Maven, PostgreSQL version).
- Add a step-by-step installation guide, database setup guide, and configuration guide.
- Add explicit run instructions and test instructions.
- Add a troubleshooting section covering common setup issues (e.g., the JDK version mismatch documented in Section 5).
- Document the supported/required Java version explicitly.
- Add Docker usage instructions, since Dockerfile and docker-compose.yml exist in the repository.
- Complete the screenshots section with actual images.
- Re-save the README as UTF-8 to correct the encoding issue.
- Align the project's stated title/description with the assigned Employee Skill-Gap Analysis and Training Recommendation System problem, or explicitly document the relationship between the two, as this report does in Section 1.



7 · Code Review Findings and Recommendations

| Area                   | Finding                                                  | Severity | Recommendation                                |
|------------------------|----------------------------------------------------------|----------|-----------------------------------------------|
| Controller             | Too many responsibilities                                | Medium   | Introduce service layer                       |
| Database configuration | Credentials embedded in source                           | High     | Externalize configuration                     |
| Exception handling     | printStackTrace() used                                   | Medium   | Structured logging / exception propagation    |
| DAO design             | Multiple responsibilities                                | Medium   | Split DAO responsibilities                    |
| Transactions           | Multi-step DB operation lacks clear transaction handling | Medium   | Use transactions / rollback                   |
| Testing                | Limited model-focused tests                              | Medium   | Add DAO / integration / controller tests      |
| Documentation          | Installation instructions limited                        | Medium   | Add setup guide                               |
| Documentation          | Assigned problem differs from README title               | Medium   | Align project description                     |
| Documentation          | Encoding problems                                        | Low      | Save README as UTF-8                          |
| Environment            | Java 23 vs Java 21 mismatch                              | Medium   | Document required JDK / version compatibility |
| Naming                 | Some abbreviations such as sofskill                      | Low      | Use descriptive names                         |



8 · Conclusion

The project demonstrates a functional foundation for employee performance management, with useful modular elements such as model classes, DAO classes, JavaFX UI components, PostgreSQL integration, and basic JUnit testing. The Git workflow applied to the repository — forking, branching, committing, opening a Pull Request, merging, and resolving a conflict — demonstrates a reasonable collaborative version-control process for a project of this scale.

However, the review identifies clear opportunities to improve separation of concerns (particularly in the controller and DAO layers), configuration security (hard-coded database credentials), exception handling, database transaction management, automated test coverage, documentation completeness, Java version compatibility documentation, and alignment between the repository's stated purpose and the assigned skill-gap/training-recommendation problem.

Addressing these recommendations would improve the project's maintainability, portability, security, reliability, testability, collaboration, and future scalability — particularly as a foundation that could be extended toward the full assigned system, which requires competency-gap scoring, personalized training recommendation, learning-path tracking, certification tracking, and HR analytics beyond what is currently documented.

## 9 · Expected Deliverables Checklist

| Deliverable                                            | Status                 | Location in Submission                             |
|--------------------------------------------------------|------------------------|----------------------------------------------------|
| Code Review Report (10–15 pages)                       | Included               | This document                                      |
| Git repository link                                    | Included               | Section 4 (clone URL)                              |
| Screenshots of repository structure and commit history | To be added by student | Figure 1 (Section 2);<br>placeholders in Section 4 |
| Code review observations with examples                 | Included               | Section 3, Section 7                               |
| Test execution screenshots                             | To be added by student | Figure 3 (Section 5)                               |
| Recommendations for improvement                        | Included               | Sections 3, 4, 5, 6, 7                             |

### Rubrics for Calculation

| Evaluation Criteria                                              | Excellent (4 Marks)                                                                                                                | Good (3 Marks)                             | Satisfactory (2 Marks)                              | Needs Improvement (1 Mark)                    |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------|-----------------------------------------------|
| <b>Problem Analysis &amp; Repository Organization(15 %)</b>      | Clearly explains the problem, solution, and repository structure with logical organization and justification.                      | Good explanation with minor omissions.     | Basic explanation and repository organization.      | Incomplete or poorly organized repository.    |
| <b>Code Quality Review(25%)</b>                                  | Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations. | Good code review with minor gaps.          | Basic review with limited analysis.                 | Superficial or inaccurate code review.        |
| <b>Version Control Evaluation(20%)</b>                           | Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.                | Good evaluation with adequate explanation. | Basic evaluation with limited evidence.             | Poor or incomplete version control analysis.  |
| <b>Testing &amp; Validation(15%)</b>                             | Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.                        | Good testing with minor omissions.         | Basic testing with limited evidence.                | Inadequate or missing testing and validation. |
| <b>Documentation &amp; Repository Maintenance(15 %)</b>          | README, installation guide, usage instructions, and documentation are complete, clear, and well organized.                         | Documentation is mostly complete.          | Basic documentation with missing details.           | Poor or insufficient documentation.           |
| <b>Review Findings, Recommendations &amp; Presentation(10 %)</b> | Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.                | Good recommendations and presentation.     | Basic recommendations with acceptable presentation. | Weak conclusions and poor presentation.       |

| Criteria                                                            | Mark |
|---------------------------------------------------------------------|------|
| System Requirement Analysis <b>(30%)</b>                            | /5   |
| Selection of Software Architecture <b>(25%)</b>                     | /5   |
| Architecture Diagram and Component Design <b>(20%)</b>              | /5   |
| Component Interaction and Quality Attribute Analysis <b>(15%)</b>   | /5   |
| Architectural Justification and Technical Presentation <b>(10%)</b> | /5   |
| <b>TOTAL</b>                                                        | /25  |